NOTES

Kerberos Protocol

AUTHOR
RootSec

VERSION
v1.0

CATEGORY
Notes

Contents

- Key Components
- The TGT and Session Keys
- The PAC (Privilege Attribute Certificate)
- S4U2Self and S4U2Proxy
- Delegation Types
  - Unconstrained Delegation
  - Constrained Delegation
  - Resource-Based Constrained Delegation (RBCD)
- Time Synchronisation
- Ticket Formats

Kerberos Protocol

Kerberos is the default authentication protocol in Active Directory, specified in RFC 4120 and extended significantly by Microsoft. It solves a fundamental problem: how can a client prove its identity to a remote service without sending its password over the network, and without requiring the service to contact a central authority every time? The answer is a system of cryptographically sealed, time-limited **tickets** issued by a trusted third party — the Key Distribution Centre (KDC), which runs on every domain controller.

The full protocol flow (AS-REQ/AS-REP, TGS-REQ/TGS-REP, AP-REQ/AP-REP) is documented with a sequence diagram in the Kerberos Encryption Types note. This note covers the protocol's components, the S4U extension mechanism (the foundation of all delegation attacks), and the krbtgt account's central role.

Key Components

The KDC (Key Distribution Centre) is a logical service running on every DC. It has two functional parts:

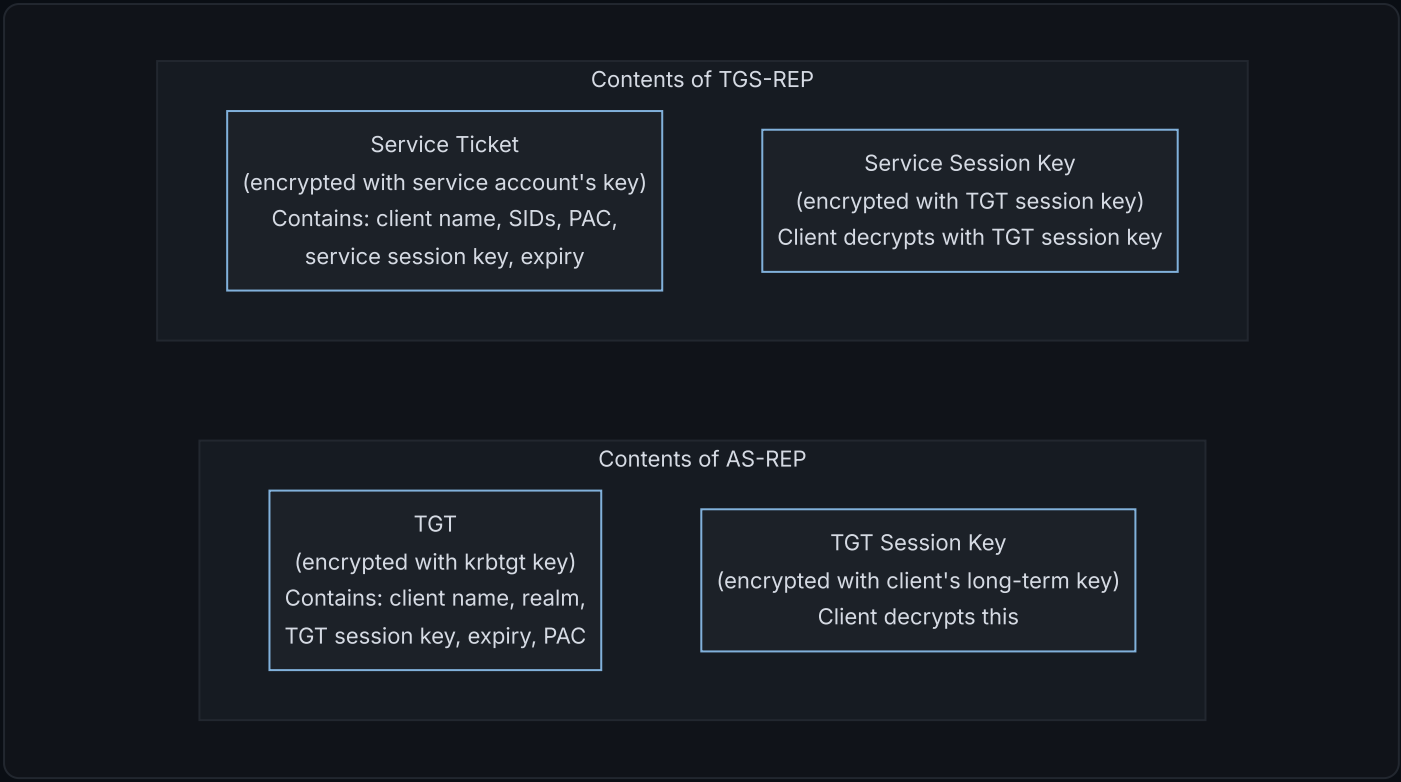
- **Authentication Service (AS)** — receives AS-REQ messages from clients proving their identity; issues TGTs in response. The AS only talks to users at login time.
- **Ticket Granting Service (TGS)** — receives TGS-REQ messages from clients who already hold a TGT; issues service tickets. The TGS runs for the duration of the user's session.

The `krbtgt` account is the KDC's own service account. Its password hash is used to encrypt and sign every TGT in the domain. The `krbtgt` hash is effectively the master key for the entire Kerberos infrastructure — if you have it, you can forge any TGT for any user, with any group memberships, valid for any duration. This is why `krbtgt` rotation is the primary remediation after a Golden Ticket attack.

Long-term keys are derived from account passwords. Every account in AD has Kerberos long-term keys stored in the NTDS.dit database. Service tickets for a given service are encrypted with that service account's long-term key. The derivation differs by etype: RC4 uses the NT hash directly; AES uses PBKDF2. See the Kerberos Encryption Types note for detail.

The TGT and Session Keys

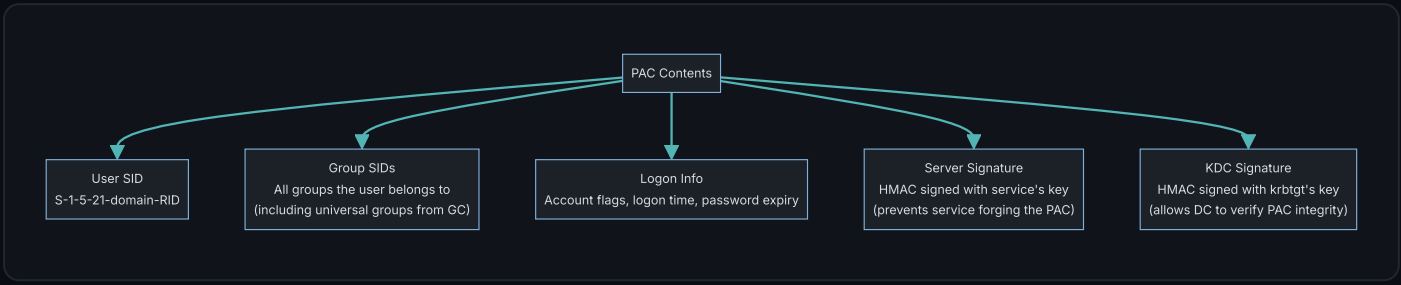
The Ticket Granting Ticket is a credential the client uses to authenticate to the TGS without re-sending its password. Crucially, the client cannot read its own TGT — it's encrypted with the `krbtgt` key, which the client does not have. The client simply stores the TGT and presents it opaquely.



The architecture of nested encryption means every secret is always wrapped in a key the recipient can read:
- The AS-REP's TGT session key is encrypted with the **client's** long-term key → only the client can decrypt it
- The TGT itself is encrypted with the **krbtgt** key → only the KDC can decrypt it (client passes it opaquely)
- The TGS-REP's service session key is encrypted with the **TGT session key** → only the client (who has the TGT session key) can decrypt it
- The service ticket is encrypted with the **service account's** key → only the service can decrypt it

The PAC (Privilege Attribute Certificate)

Every Kerberos ticket (TGT and service ticket) contains a **PAC** embedded in the `authorization-data` field. The PAC is a Microsoft extension to the Kerberos spec that carries the user's identity and group memberships for use in access control decisions.



The KDC signature is the critical anti-forgery control. Without the krbtgt key, an attacker cannot forge a valid PAC. With the krbtgt key (Golden Ticket), they can forge any PAC content — including adding themselves to Enterprise Admins.

Services can validate the PAC in two ways:
1. **Direct validation**: verify the server signature locally (uses only the service's own key — vulnerable to Silver Ticket if the service account's key is compromised)
2. **NetLogon validation**: call `NetrLogonSamLogon()` to the DC, which verifies the KDC signature (catches Silver Tickets, since the DC holds the krbtgt key for KDC signature verification)

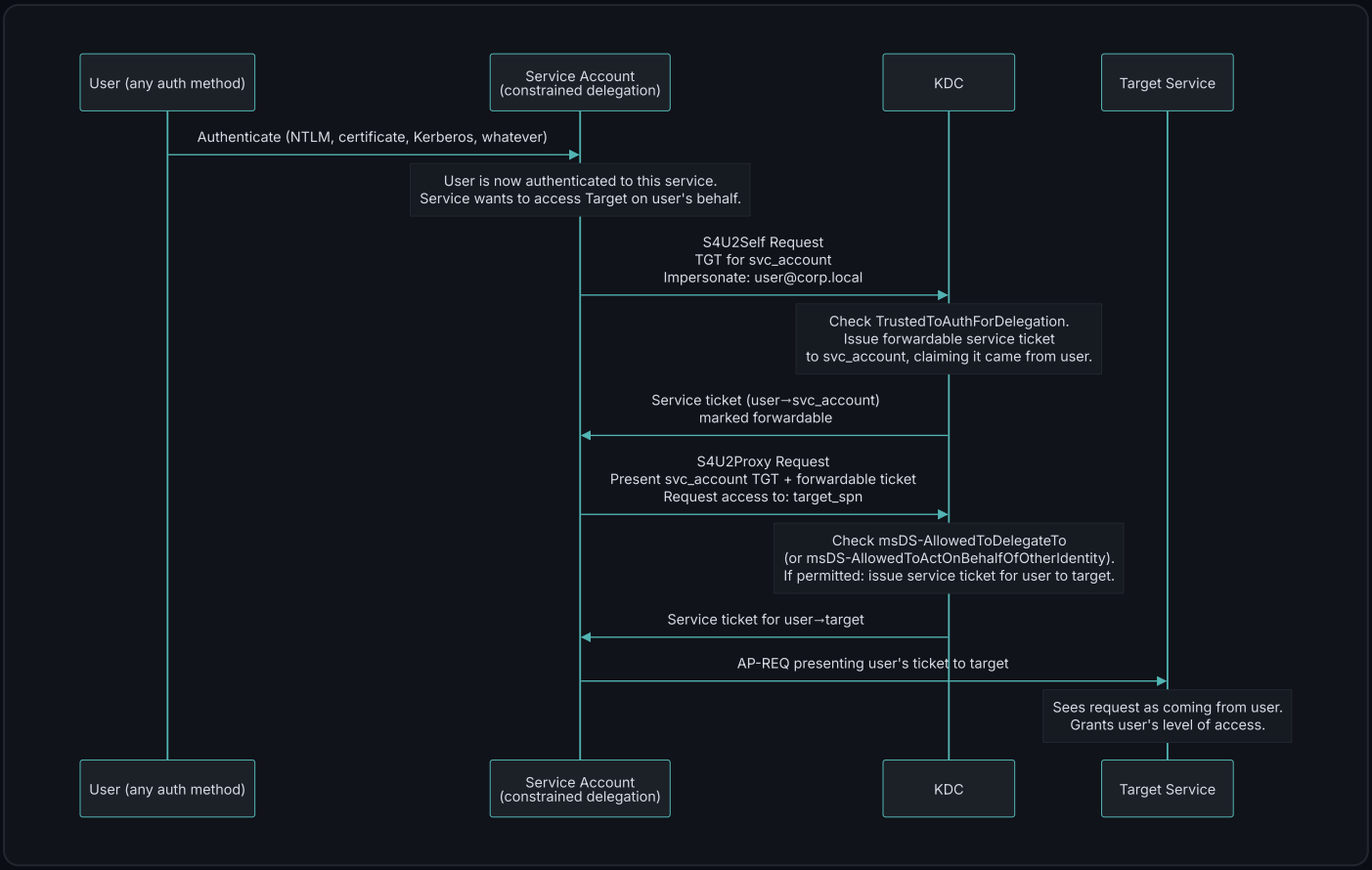
Most services use direct validation for performance. This is why Silver Tickets work — services don't call home by default.

S4U2Self and S4U2Proxy

These are Microsoft Kerberos extensions that implement **protocol transition** and **constrained delegation**. They allow a service to request Kerberos tickets on behalf of a user — even if that user authenticated via NTLM, a certificate, or some non-Kerberos mechanism.

S4U2Self (Service for User to Self): A service requests a service ticket to itself, impersonating a named user. The KDC issues this ticket without requiring the user's TGT or any user involvement. The ticket appears to come from the specified user, allowing the service to act on their behalf. This requires the service account to have `TrustedToAuthForDelegation` set (protocol transition enabled).

S4U2Proxy (Service for User to Proxy): Takes the S4U2Self ticket (or a user-provided ticket from Kerberos authentication) and uses it to request a service ticket to a **third** service on the user's behalf. The KDC verifies that the requesting service is permitted to delegate to the target SPN (via `msDS-AllowedToDelegateTo` or `msDS-AllowedToActOnBehalfOfOtherIdentity`).



Delegation Types

Unconstrained Delegation

When a computer has `TrustedForDelegation=TRUE` set, the DC embeds the user's actual TGT (not just a derived S4U ticket) inside the service ticket sent to that machine. The KDC sets the `ok-as-delegate` flag and includes the user's full TGT as `authorization-data` in the ticket. When the service decrypts its service ticket, it extracts this embedded TGT and caches it in LSASS.

This means any user who authenticates to an unconstrained machine leaves a copy of their TGT on that machine, accessible to anyone with SYSTEM access. Domain Controllers themselves use unconstrained delegation (they need to impersonate users freely) — which is why enumerating non-DC unconstrained machines is an attack target.

Constrained Delegation

The service account has `msDS-AllowedToDelegateTo` populated with a list of permitted target SPNs. The S4U2Proxy call succeeds only if the target SPN is in this list. The attack target: compromise the delegating account → S4U2Self to impersonate any user → S4U2Proxy to any permitted SPN.

Resource-Based Constrained Delegation (RBCD)

Instead of the source account being configured for delegation, the **target resource** stores which principals can delegate to it in `msDS-AllowedToActOnBehalfOfOtherIdentity`. This reverses the trust relationship — the target decides who can impersonate users to access it, rather than the source being trusted by the KDC.

The attack consequence: if you can write `msDS-AllowedToActOnBehalfOfOtherIdentity` on a machine object (via GenericWrite, WriteDACL, etc.), you can add any computer account you control, then S4U2Self + S4U2Proxy to impersonate any user to that machine.

Time Synchronisation

Kerberos rejects tickets where the timestamp differs from the KDC's clock by more than **5 minutes**. This prevents replay attacks — a captured Authenticator (timestamp encrypted with session key) is only valid for 5 minutes. On Linux systems attacking AD: sync time before attempting Kerberos operations.

```
# Sync time to DC
sudo ntpdate <DC_IP>

# Or use faketime for a one-off adjustment
faketime -f "+3600" impacket-getTGT corp.local/user:'Password1'

# Impacket tools have -no-pass and will auto-adjust if kerberostime is set
```

Ticket Formats

FORMAT	EXTENSION	USED BY
KRB credentials cache	`.ccache`	Linux, Impacket, MIT Kerberos
KRB credentials (binary)	`.kirbi`	Windows (Mimikatz, Rubeus)

Convert between formats:

```
# ccache → kirbi
impacket-ticketConverter ticket.ccache ticket.kirbi

# kirbi → ccache
impacket-ticketConverter ticket.kirbi ticket.ccache

# Rubeus base64 kirbi → ccache (via decode + convert)
base64 -d rubeus_output.b64 > ticket.kirbi
impacket-ticketConverter ticket.kirbi ticket.ccache
```

Use a ticket on Linux:

```
export KRB5CCNAME=/path/to/ticket.ccache
impacket-psexec -k -no-pass corp.local/Administrator@dc.corp.local
```